Gauss Hermite Fourier Features Based on Maximum Correntropy Criterion for Adaptive Filtering

Tian Zhou, Shiyuan Wang, *Senior Member, IEEE*, Junhui Qian, *Member, IEEE*, Yunfei Zheng, Qiangqiang Zhang, *Graduate Student Member, IEEE*, and Yingying Xiao

***Abstract*— Kernel adaptive filters (KAFs) are a class of nonlinear adaptive filters developed in the reproducing kernel Hilbert space, and are particularly suitable for addressing signal processing issues involving data streams and unknown nonlinearities. However, KAFs endure the issue of network structure growth as the number of training samples increases. To this end, a novel structural sparsification method for KAFs, i.e., Gauss Hermite Fourier features (GHFF) method, is first proposed by combining Gauss Hermite quadrature integration rule and Fourier transform. Subsequently, the GHFF method is integrated with the maximum correntropy criterion in filter design, leading to the development of two new adaptive filtering algorithms, i.e., GHFF maximum correntropy (GHFFMC) algorithm and stochastic batch GHFFMC (SB-GHFFMC) algorithm. The proposed GHFFMC and SB-GHFFMC algorithms are expected to exhibit excellent capabilities in characterizing the unknown nonlinear relationships within the data, along with robustness to outliers. Meanwhile, SB-GHFFMC is anticipated to exhibit superior filtering performance in comparison with GHFFMC, as it leverages a general and flexible batch gradient descent method for model optimization. Simulations on nonlinear system identification and time-series prediction of Chua's circuits confirm the performance superiorities of the proposed algorithms compared to other robust KAFs and RFF-based filters.**

***Index Terms*— Gauss quadrature, random Fourier features, Gauss Hermite quadrature, maximum correntropy criterion, stochastic batch.**

I. Introduction

THE modelling process of nonlinear circuits presents challenges in terms of distortion, complex dynamic behaviour, and robustness [1], [2]. Techniques such as nonlinear modelling, system identification, and kernel adaptive filters (KAFs) [3], [4], have been widely adopted to cope with these challenges. KAFs integrate kernel methods with adaptive filters [5] and leverage the nonlinear modeling capability of kernel method to map input signals into high-dimensional features spaces, thereby enabling the resolution of linearly inseparable problems in low-dimensional spaces. Therefore, KAFs showcase robust nonlinear modeling capabilities, facilitating the processing of complex signals and systems [6]. They are well-suited for modeling and predicting nonlinear systems. Additionally, KAFs exhibit adaptability to real-time input data in non-stationary environments [7].

Kernel methods depend on the kernel function, empowering KAFs to compute inner products in the reproducing kernel Hilbert space (RKHS) [8] without the need for explicit features space transformation. Typical KAFs include kernel least mean square (KLMS) [9], [10] algorithm and kernel affine projection (KAPA) [3] algorithm, etc. While kernel methods for computing inner products in RKHS offer powerful capabilities for nonlinear signal processing, they often induce the issue of network structure growth as the number of training samples increases. This poses a significant challenge in scenarios where computational and storage resources are constrained, e.g., a digital signal processor with limited physical memory [11]. To reduce the computation and storage costs while preserving robust nonlinear processing capabilities, approximating kernel functions is crucial. It provides a practical solution for expanding kernel methods across diverse application scenarios [12].

Unlike KAFs, the random Fourier features (RFF) approach [13], [14], [15] maps data into a finite-dimensional RFF space through random mapping. It approximates the inner product in the RFF space to the inner product in the low-dimensional feature space by leveraging the properties of the Fourier transform, effectively reducing network structural complexity through random mapping. When the kernel function in KAFs exhibits shift invariance, RFF mapping can effectively approximate the kernel function [16], [17]. Based on RFF, the random Fourier features least mean square (RFFKLMS) [4] algorithm and the random Fourier features maximum correntropy (RFFMC) [18] algorithm have been proposed successively. However, the mapping process of the RFF method can lead to inaccurate approximations of kernel functions due to its reliance on random samples. Although this imprecision can be reduced by employing a large number of RFF features, it may necessitate higher computational complexity.

To offer an alternative solution to the RFF method for the approximation of kernel functions, this paper employs

Received 13 May 2024; revised 24 July 2024 and 15 August 2024; accepted 1 October 2024. Date of publication 16 October 2024; date of current version 27 February 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62071391 and Grant 62306245 and in part by Chongqing Postdoctoral Science Foundation Special Funded under Grant 2022CQBSHTB3076. This article was recommended by Associate Editor X. Zeng. *(Corresponding author: Shiyuan Wang.)*

Tian Zhou, Shiyuan Wang, Yunfei Zheng, Qiangqiang Zhang, and Yingying Xiao are with the College of Electronic and Information Engineering, Southwest University, Chongqing 400715, China (e-mail: zt1395660832@email.swu.edu.cn; wsy@swu.edu.cn; zhengyfswu@swu.edu.cn; zqqfilter@email.swu.edu.cn; xyy2021@email.swu.edu.cn).

Junhui Qian is with the School of Microelectronic and Communication Engineering, Chongqing University, Chongqing 400044, China, and also with Chongqing Key Laboratory of Bio-Perception and Intelligent Information Processing, Chongqing 400044, China (e-mail: junhuiq@cqu.edu.cn).

Digital Object Identifier 10.1109/TCSI.2024.3476150

quadrature methods to construct mapping features. Since this approach allows for a relatively accurate calculation of the quadrature points and their corresponding weight coefficients for approximation of kernel functions [19], [20], an improved accuracy can be expected. Further, for a typical Gaussian kernel function, the integration interval as well as the weight function of the Gauss Hermite quadrature method are consistent with the calculation of the Gaussian kernel function. Hence, Gauss Hermite Fourier features (GHFF) [21] mapping is used to approximate the Gaussian kernel function in this paper.

The loss function has a significant impact on the performance of KAFs. The mean square error (MSE) [22], as a commonly used loss function, is susceptible to the interference of outliers. To enhance the robustness of KAFs, the correntropy in information theory learning (ITL) [23] is introduced as the cost function, where the minimum error entropy (MEE) [24] method imposes a greater computational burden compared to the maximum correntropy criterion (MCC) [25], [26], [27], [28], [29], [30]. Therefore, MCC is widely used in filtering algorithms due to its low computational complexity. A classical application is the RFFMC [18] algorithm, which combines RFF and MCC, showcasing excellent robustness and filtering performance. In this paper, the MCC is employed as the cost function for filtering, and it is subsequently integrated with the GHFF mapping to yield the GHFF maximum correntropy (GHFFMC) algorithm.

Although the RFFMC algorithm effectively reduces the computational burden of the kernel maximum correntropy (KMC) [31] algorithm, its performance remains inferior to that of the KMC algorithm. This paper proposes the GHFF mapping as an alternative to RFF and the GHFFMC algorithm is therefore proposed in combination with MCC in order to achieve a similar performance as the KMC algorithm. Moreover, to achieve higher filtering performance than the GHFFMC and KMC algorithms, we propose the stochastic batch GHFFMC (SB-GHFFMC) algorithm. Since both the current input and multiple previous samples are synchronously selected for updating the filtering weights, SB-GHFFMC is expected to capture more useful information hidden in data compared to GHFFMC and KMC algorithms at each iteration, thereby further improving the filtering performance. The main contributions of this paper are given as follows:

1) The GHFF mapping method is proposed based on Gauss Hermite numerical quadrature rule to approximate the Gaussian kernel function well with relatively low extended dimensions.

2) The GHFFMC algorithm is proposed to achieve the equivalent performance of the KMC algorithm with much lower computational complexity. Further, to enhance the performance of GHFFMC and KMC, the SB-GHFFMC algorithm is designed by combining the fast convergence of stochastic gradient with the stability of batch gradient [32], [33].

3) The expressions of steady-state mean-square-deviation (MSD) for the proposed GHFFMC and SB-GHFFMC algorithms are derived for performance analysis, and the corresponding theoretical validations by simulations are provided.

The organization of this paper is given as follows. Section II introduces the basic concepts of RFF and Gauss Hermite quadrature. In Section III, the proposed GHFFMC and SB-GHFFMC algorithms are derived. Section IV provides a theoretical analysis regarding the proposed algorithms. In Section V, simulations are used to validate the performance superiorities of the proposed algorithms. Section VI gives the conclusion.

II. Preliminaries

A. Random Fourier Features

The RFF method [34] has been proved to be effective to curb the growth of network structure of kernel methods, as it directly maps the input data from the original space into a limited and low dimensional feature space rather than RKHS.

For inputs $\boldsymbol{x}$ and $\boldsymbol{y}$, the commonly used kernel function is the Gaussian kernel function, which is defined as

$$\kappa(\boldsymbol{x}, \boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|^2}{2\sigma^2}\right), \tag{1}$$

where $\sigma > 0$ is the kernel bandwidth. It is clear that the Gaussian kernel is a shift-invariant kernel.

According to Bochner's theorem [13], a continuous kernel $\kappa(\boldsymbol{x}, \boldsymbol{y}) = \kappa(\boldsymbol{x}-\boldsymbol{y})$ with shift invariance can be expressed in Fourier transform form. Thus, (1) can be rewritten as

$$\kappa(\boldsymbol{x}, \boldsymbol{y}) = \int_{-\infty}^{+\infty} p(\boldsymbol{\omega}) \exp\left(j\boldsymbol{\omega}^T(\boldsymbol{x}-\boldsymbol{y})\right) d\boldsymbol{\omega}, \tag{2}$$

where $p(\boldsymbol{\omega})$ is the Fourier transform of $\kappa(\boldsymbol{x}, \boldsymbol{y})$ and j is an imaginary unit.

When considering only real inputs, (2) can be further written as follows

$$\begin{aligned}\kappa(\boldsymbol{x}, \boldsymbol{y}) &= \int_{-\infty}^{+\infty} p(\boldsymbol{\omega}) \cos\left(\boldsymbol{\omega}^T(\boldsymbol{x}-\boldsymbol{y})\right) d\boldsymbol{\omega} \\ &\approx \boldsymbol{\Phi}_{\boldsymbol{\omega}}{}^T(\boldsymbol{x})\, \boldsymbol{\Phi}_{\boldsymbol{\omega}}(\boldsymbol{y}),\end{aligned} \tag{3}$$

where the unbiased estimate of $\kappa(\boldsymbol{x}, \boldsymbol{y})$ is $\boldsymbol{\Phi}_{\boldsymbol{\omega}}{}^T(\boldsymbol{x})\, \boldsymbol{\Phi}_{\boldsymbol{\omega}}(\boldsymbol{y})$, $(\cdot)^T$ denotes the transposition operator of a matrix or vector.

The RFF mapping can be written as

$$\boldsymbol{\Phi}_{\boldsymbol{\omega}}(\boldsymbol{x}) = \sqrt{\frac{2}{D}}\left[\cos\left(\boldsymbol{\omega}_1^T\boldsymbol{x}+b_1\right), \cdots, \cos\left(\boldsymbol{\omega}_D^T\boldsymbol{x}+b_D\right)\right]^T, \tag{4}$$

where the vector of $\boldsymbol{\omega}_i \in \{\boldsymbol{\omega}_1, \cdots, \boldsymbol{\omega}_D\} = \boldsymbol{\omega}$ is sampled from the distribution of $p(\boldsymbol{\omega})$ [14], and b_i is a random variable set sampled from uniform distribution over $(0, 2\pi)$ [35].

B. Numerical Quadrature and Gauss Hermite Quadrature

For any function F, to solve the definite integral $\int_{\bar{a}}^{\bar{b}} F(\omega)d\omega$ in the form of (2), we consider using Gaussian quadrature [36] to approximate the kernel function. In the numerical quadrature method, we can appropriately select some nodes ω_i on the interval $[\bar{a}, \bar{b}]$, thus $F(\omega_i)$ is a_i-weighted and averaged to achieve an approximation of F, i.e.,

$$\int_{\bar{a}}^{\bar{b}} F(\omega)d\omega \approx \sum_{i=1}^{D} a_i F(\omega_i). \tag{5}$$

A quadrature formula of the form (5) with the highest algebraic accuracy of $2D+1$ is called a Gauss-type quadrature formula [37]. To make the problem more general, we consider the following quadrature with weights

$$\int_{\bar{a}}^{\bar{b}} W(\omega)F(\omega)d\omega \approx \sum_{i=1}^{D} a_i F(\omega_i), \tag{6}$$

where $W(\omega)$ is a weight function. The difference of weight function $W(\omega)$ will lead to different Gaussian quadrature rules for finding the Gaussian quadrature points $\{\omega_i\}_{i=1}^{D}$ and the corresponding weight coefficients $\{a_i\}_{i=1}^{D}$.

For finding Gaussian quadrature points, the following conditions should be satisfied.

1) Gaussian quadrature points $\{\omega_i\}_{i=1}^{D}$ are the zero-points of a $D+1$-order polynomial $Q_{D+1}(\omega)$.

2) In the interval of quadrature, the polynomial $Q_{D+1}(\omega)$ is orthogonal to any polynomial $P(\omega)$ of order up to D with weight $W(\omega)$, that is

$$\int_{\bar{a}}^{\bar{b}} P(\omega)Q_{D+1}(\omega)W(\omega)d\omega = 0. \tag{7}$$

With the obtained Gaussian quadrature points, the corresponding weight coefficients $\{a_i\}_{i=1}^{D}$ can be obtained according to the selected Gaussian quadrature rules [38], i.e.,

$$a_i = \int_{\bar{a}}^{\bar{b}} W(\omega)\frac{t(\omega)}{(\omega-\omega_i)t'(\omega_i)}d\omega, \tag{8}$$

where $t(\omega) = \prod_{j=1}^{D}(\omega-\omega_j)$ is an interpolating polynomial of order D with $2D+1$ algebraic accuracy used to approximate the integrand $F(\omega)$, and t' is the first-order derivative of t.

In Gaussian quadrature formula (6), if the weight function $W(\omega)$ is $\exp(-\omega^2)$ and the quadrature interval is $(-\infty,+\infty)$, the Gauss Hermite quadrature (GHQ) formula can be written as [38]

$$\int_{-\infty}^{+\infty}\exp(-\omega^2)F(\omega)d\omega \approx \sum_{i=1}^{D} h_i F(\omega_i) + E_H, \tag{9}$$

where $E_H = \frac{D!\sqrt{\pi}}{2^D(2D)!}f^{(2D)}(\xi)$ [36] denotes the remainder term. $\{\omega_i\}_{i=1}^{D}$ are the zero-points of the Hermite polynomial $H(\omega)$ of order D and obey the standard normal distribution, and $\{h_i\}_{i=1}^{D}$ are the weight coefficients. The Hermite polynomial expression of order D is thus given by [39]

$$H_D(\omega) = (-1)^D\exp(\omega^2)\frac{d^D}{d\omega^D}\exp(-\omega^2), \tag{10}$$

which satisfies the 3-term recurrence relationship as follows

$$H_{i+1}(\omega) = (\lambda_{1,i}\omega+\lambda_{2,i})H_i(\omega) - \lambda_{3,i}H_{i-1}(\omega), \tag{11}$$

where $\lambda_{1,i}=2$, $\lambda_{2,i}=0$, and $\lambda_{3,i}=2i$.

In this case, the interpolation polynomial $t(\omega)$ in (8) is $\frac{H_D(\omega)}{2^D}$. Then the weight coefficients $\{h_i\}_{i=1}^{D}$ can be expressed as

$$h_i = \frac{2^{D+1}D!\sqrt{\pi}}{(H_{D+1}(\omega_i))^2}, i=1,\cdots,D. \tag{12}$$

Algorithm 1 Golub-Welsch Algorithm for Calculating Gauss-Hermite Quadrature Points and Weight Coefficients

Input: Number of quadrature nodes D.
Calculation:
(1) Compute the 3-term recurrence coefficients $\{\gamma_i\}_{i=1}^{D}$ and $\{\beta_i\}_{i=1}^{D-1}$ using (14);
(2) Construct the Gauss Hermite tridiagonal matrix $\boldsymbol{J}$ using (13);
(3) Find the eigenvalues of matrix $\boldsymbol{J}$ that are the quadrature nodes $\{\omega_i\}_{i=1}^{D}$;
(4) Calculate the weight coefficients $\{h_i\}_{i=1}^{D}$ using (15).
end

However, (12) is complicated by directly calculating the zero-points $\{\omega_i\}_{i=1}^{D}$ of the Hermite polynomial as well as solving the weight coefficients $\{h_i\}_{i=1}^{D}$. Therefore, by applying the polynomial coefficients of (11), we can construct the Hermite tridiagonal matrix $\boldsymbol{J}$, i.e.,

$$\boldsymbol{J} = \begin{bmatrix} \gamma_1 & \beta_1 & 0 & \cdots & 0 \\ \beta_1 & \gamma_2 & \beta_2 & \ddots & \vdots \\ 0 & \beta_2 & \ddots & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & \beta_{D-1} \\ 0 & \cdots & 0 & \beta_{D-1} & \gamma_D \end{bmatrix}, \tag{13}$$

where the 3-term recurrence coefficients $\{\gamma_i\}_{i=1}^{D}$ and $\{\beta_i\}_{i=1}^{D-1}$ are calculated by

$$\begin{cases} \gamma_i = -\dfrac{\lambda_{2,i}}{\lambda_{1,i}} = 0, & i=1,\cdots,D \\ \beta_i = \sqrt{\dfrac{\lambda_{3,i+1}}{\lambda_{1,i}\lambda_{1,i+1}}}, & i=1,\cdots,D-1. \end{cases} \tag{14}$$

Thus, we employ the Golub-Welsch [40] algorithm to obtain the eigenvalues and eigenvectors of $\boldsymbol{J}$ to determine the quadrature points and their corresponding weight coefficients, where the eigenvalues represent the quadrature points ω_i. As a consequence of the Christoffel-Darboux identity, (12) satisfies $h_i\boldsymbol{p}_{\omega_i}^T\boldsymbol{p}_{\omega_i}=1$, where $\boldsymbol{p}_{\omega_i}$ denotes the eigenvector corresponding to the eigenvalue ω_i of $\boldsymbol{J}$. Thus, the weight coefficients can be derived as

$$\begin{aligned} h_i &= \frac{\boldsymbol{p}_{\omega_i}^2(1)}{H_0^2(\omega_i)} \\ &= \boldsymbol{p}_{\omega_i}^2(1)\int_{-\infty}^{\infty}\exp(\omega^2)d\omega \\ &= \sqrt{\pi}\,\boldsymbol{p}_{\omega_i}^2(1), i=1,\cdots,D \end{aligned} \tag{15}$$

The Golub-Welsch algorithm for calculating Gauss-Hermite quadrature points and their weight coefficients is summarized in Algorithm 1.

III. Proposed Algorithms

A. Gauss Hermite Fourier Features

In this section, we will provide the details regarding how to use the GHQ rule to design new randomized mapping features.

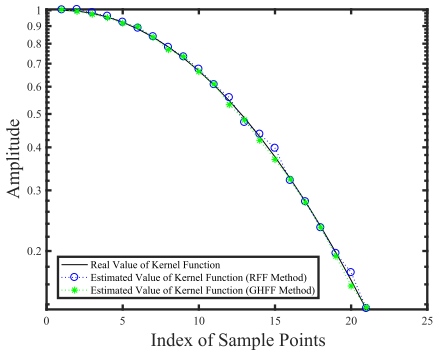


Fig. 1. Comparison between the original kernel function values and the kernel function values estimated using GHFF and RFF methods.

However, since the GHQ rule is primarily designed for one-dimensional data, we will start by discussing its application in one-dimensional scenarios. Next, we will discuss the extension of this approach to handle multi-dimensional data.

1) Gauss Hermite Fourier Features for One-Dimensional Data: Assume that x and y are two arbitrarily random variables of one-dimensional. Similar to (2), the kernel metric between them can be given by

$$\kappa(x, y) = \int_{-\infty}^{+\infty} \exp\left(-\omega^2\right) \exp\left(j\omega \frac{\sqrt{2}(x-y)}{\sigma}\right) d\omega. \tag{16}$$

Further, since the accuracy of approximating a Gaussian kernel function using GHFF primarily depends on the selection of D_p quadrature points and their weight coefficients, (16) can be rewritten as

$$\int_{-\infty}^{+\infty} \exp\left(-\omega^2\right) \exp\left(j\omega \frac{\sqrt{2}(x-y)}{\sigma}\right) d\omega \approx \sum_{i=l}^{l+D_p} h_i \exp\left(j\omega_i \frac{\sqrt{2}(x-y)}{\sigma}\right). \tag{17}$$

It should be noted that, as the majority of weight coefficients remain near zero, the mapping form exhibits a significant proportion of values tending towards zero. Therefore, we can reuse the valid partial quadrature points and their corresponding weight coefficients within the quadrature interval to construct new quadrature points and weight coefficients. The reconstructed weight coefficients are

$$\frac{D_p}{D}\left[\underbrace{h_l, \cdots h_{l+D_p}}_{1}, \underbrace{h_l, \cdots h_{l+D_p}}_{2}, \cdots, \underbrace{h_l, \cdots h_{l+D_p}}_{D/D_p}\right], \tag{18}$$

where $D_p \le D$, and $\left[h_l, \cdots, h_{l+D_p}\right]$ are reused D/D_p times. Meanwhile, the weight coefficients are reused in a similar way to maintain consistency. The choice of D_p depends on the magnitude of the weight coefficients in the Gauss-Hermite solution. Specifically, for small values of D, the weight coefficients are generally non-zero, and thus we set $D_p = D$. However, for relatively large values of D (e.g., $D = 100$), some weight coefficients will tend towards zero. In such cases, D_p will be manually adjusted to retain only non-zero points, thereby preventing the retention of redundant elements and promoting the desired filtering performance.

When considering only the real number case, the GHFF can be rewritten by combining (9) and (16), i.e.,

$$\kappa(x, y) = \int_{-\infty}^{+\infty} \exp\left(-\omega^2\right) \cos\left(\omega \frac{\sqrt{2}(x-y)}{\sigma}\right) d\omega \approx \sum_{i=1}^{D} h_i \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}(x-y)\right). \tag{19}$$

If we set $b \sim U(0, 2\pi)$, by utilizing the periodicity property of the cosine function, it can be deduced that

$$\begin{aligned} &E_\omega\left[\cos\left(\omega \frac{\sqrt{2}}{\sigma}(x+y) + 2b\right)\right] \\ &\quad = E_\omega\left[E_b\left[\cos\left(\omega \frac{\sqrt{2}}{\sigma}(x+y) + 2b\right)|\omega\right]\right] \\ &\quad = 0, \end{aligned} \tag{20}$$

where E denotes the expectation operator and $E_b[\cdot|\omega]$ denotes the expectation of b, when ω is known.

Combining (20) and using the GHQ method, we have

$$\begin{aligned} &\sum_{i=1}^{D} h_i \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}(x+y) + 2b_i\right) \\ &\quad \approx E_\omega\left[\cos\left(\omega \frac{\sqrt{2}}{\sigma}(x+y) + 2b\right)\right] \\ &\quad = 0. \end{aligned} \tag{21}$$

Therefore, (19) can be rewritten as

$$\begin{aligned} &\sum_{i=1}^{D} h_i \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}(x-y)\right) \\ &= \sum_{i=1}^{D} h_i \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}(x-y)\right) + \sum_{i=1}^{D} h_i \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}(x+y) + 2b_i\right) \\ &= \sum_{i=1}^{D} \sqrt{2h_i} \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}x + b_i\right) \sqrt{2h_i} \cos\left(\omega_i \frac{\sqrt{2}}{\sigma}y + b_i\right) \\ &= \boldsymbol{\Phi}^T(x)\, \boldsymbol{\Phi}(y), \end{aligned} \tag{22}$$

where $\boldsymbol{\Phi}(x)$ is the new representation of the original x with GHFF method, and is constructed by

$$\boldsymbol{\Phi}(x) = \begin{bmatrix} \sqrt{2h_1} \cos\left(\omega_1 \frac{\sqrt{2}}{\sigma}x + b_1\right) \\ \vdots \\ \sqrt{2h_D} \cos\left(\omega_D \frac{\sqrt{2}}{\sigma}x + b_D\right) \end{bmatrix}. \tag{23}$$

To clearly demonstrate the efficiency of using GHFF to approximate a user-specified kernel function, Fig. 1 presents a simple example that adopts GHFF method to approximate a Gaussian kernel with the kernel size of 1. For comparison, the estimated results of the kernel function with RFF method are also provided. Herein, the values of expanded dimension D for GHFF and RFF methods are both set to 20, and the value of D_p for GHFF method is set to 10. As can be seen

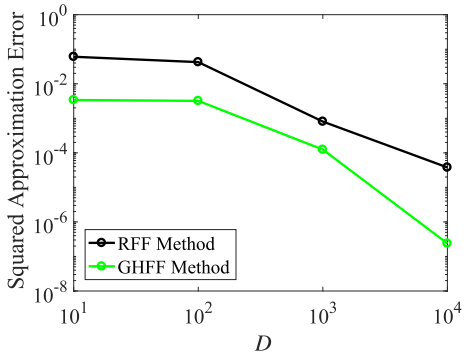


Fig. 2. Approximation errors of GHFF and RFF for a Gaussian kernel function under different extended dimensions.

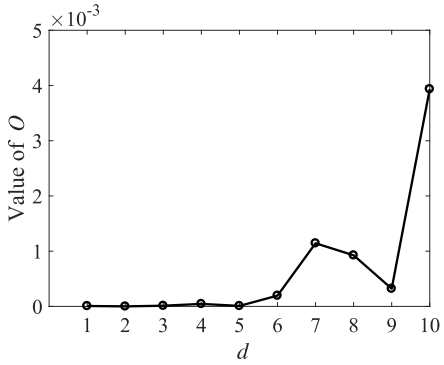


Fig. 3. The value of O versus d.

from Fig. 1, the value of the estimated kernel function with the GHFF method is closer to the real one, which means that GHFF can approximate the target function more accurately than RFF in the following simulations. In addition, the squared approximation errors of GHFF and RFF methods for the approximation of a Gaussian kernel function under different expansion dimensions are shown in Fig. 2. It can be seen from Fig. 2 that, the squared approximation error of GHFF method with limited dimension is generally smaller than that of RFF method.

2) Gauss Hermite Fourier Features for Multi-Dimensional Data: We now discuss the application of the GHQ rule to multi-dimensional data scenarios. In particular, let $\boldsymbol{x} = [x_1, \cdots, x_d]^T$ and $\boldsymbol{y} = [y_1, \cdots, y_d]^T$ denote two d-dimensional random variables. By utilizing the multiplication operation rule of exponential functions, the Gaussian kernel in (1) can be rewritten as

$$\exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|^2}{2\sigma^2}\right) = \prod_{m=1}^{d} \exp\left(-\frac{(x_m - y_m)^2}{2\sigma^2}\right). \tag{24}$$

For each dimension of data, there are $\{\omega_i\}_{i=1}^{D}$ and $\{h_i\}_{i=1}^{D}$ that can be obtained using Algorithm 1. Thus, for d-dimensional data, there exist $d \times D$ quadrature points and weight coefficients, i.e., $\{\omega_{m,i}\}_{m,i=1}^{d,D}$ and $\{h_{m,i}\}_{m,i=1}^{d,D}$. According to the tensor product extension quadrature rule, an extended form of (19) becomes

$$\begin{aligned}\kappa(\boldsymbol{x}, \boldsymbol{y}) &= \int_{-\infty}^{+\infty} \exp\left(-\|\boldsymbol{\omega}\|^2\right) \cos\left(\boldsymbol{\omega}^T \frac{\sqrt{2}(\boldsymbol{x}-\boldsymbol{y})}{\sigma}\right) d\boldsymbol{\omega} \\ &\approx \prod_{m=1}^{d} \sum_{i=1}^{D} h_{m,i} \cos\left(\omega_{m,i} \frac{\sqrt{2}(x_m - y_m)}{\sigma}\right) \\ &= \sum_{i=1}^{D} \tilde{h}_i \cos\left(\tilde{\boldsymbol{\omega}}_i^T \frac{\sqrt{2}}{\sigma}(\boldsymbol{x}-\boldsymbol{y})\right) + O, \end{aligned} \tag{25}$$

where $\tilde{\boldsymbol{\omega}}_i = \left[\omega_{1,i}, \omega_{2,i}, \cdots, \omega_{d,i}\right]^T$, $\tilde{h}_i$ is the normalized value of $\prod_{m=1}^{d} h_{m,i}$ with $i = 1, 2, \cdots, D$, and O represents the sum of the products of the remaining cross terms. Assuming that the value of O is negligible and using a similar process to (20)-(23), we obtain the final mapping features of $\boldsymbol{x}$ as

$$\boldsymbol{\Phi}(\boldsymbol{x}) = \begin{bmatrix} \sqrt{2\tilde{h}_1} \cos\left(\tilde{\boldsymbol{\omega}}_1^T \frac{\sqrt{2}}{\sigma}\boldsymbol{x} + b_1\right) \\ \vdots \\ \sqrt{2\tilde{h}_D} \cos\left(\tilde{\boldsymbol{\omega}}_D^T \frac{\sqrt{2}}{\sigma}\boldsymbol{x} + b_D\right) \end{bmatrix}. \tag{26}$$

Remark 1: It should be noted that the derivation of (26) is based on the assumption that the value of O is negligible. Therefore, as the value of O increases, the accuracy of the obtained results will decrease, potentially leading to an imprecise approximation of a Gaussian kernel function. However, if the dimensionality of data, i.e., d, is set to be relatively small, this assumption can always be satisfied. For instance, when setting $d = 1$, the error term O becomes 0. Further, according to the simulation results shown in Fig. 3, the value of O remains exceptionally close to 0 when we set $d \leq 5$.

B. The GHFFMC Algorithm

To design an adaptive learning algorithm, the loss function is of great importance. Typically, the MSE loss is a default choice. Although it can be a good candidate to address the issues in Gaussian noise environments, it is extremely sensitive to outliers. To offer an alternative solution to robust learning, a popular similarity measure called correntropy [25] is adopted to design the loss function, i.e.,

$$J_1(i) = \exp\left(-\frac{e^2(i)}{2\sigma_m^2}\right), \tag{27}$$

where $e(i) = d(i) - \boldsymbol{\Omega}^T(i-1)\boldsymbol{\Phi}(\boldsymbol{x}(i))$ denotes the error at time i, $d(i)$ is the desired signal, $\boldsymbol{\Omega}(i-1)$ is weight vector and σ_m represents the kernel width in correntropy.

By utilizing the popular stochastic gradient descent (SGD) method, the update equation of weights in the GHFFMC algorithm can be given by

$$\begin{aligned}\boldsymbol{\Omega}(i) &= \boldsymbol{\Omega}(i-1) + \mu \frac{\partial J_1(i)}{\partial \boldsymbol{\Omega}(i-1)} \\ &= \boldsymbol{\Omega}(i-1) + \mu \exp\left(-\frac{e^2(i)}{2\sigma_m^2}\right) e(i) \boldsymbol{\Phi}(\boldsymbol{x}(i)) \\ &= \boldsymbol{\Omega}(i-1) + \mu\varphi(i)\boldsymbol{\Phi}(\boldsymbol{x}(i)), \end{aligned} \tag{28}$$

where $\varphi(i) = \varphi(e(i)) = \exp\left(-\frac{e^2(i)}{2\sigma_m^2}\right) e(i)$, and μ denotes the step size.

C. Stochastic Batch GHFFMC Algorithm

The GHFFMC algorithm only uses the current input data to update the parameters at each iteration, and the historical information regarding the data is ignored. In this section, to further improve the filtering performance, an extended loss function is therefore designed to further utilize the historical information hidden in the data [41], generating the SB-GHFFMC algorithm. Specifically, the new loss function is given by

$$J_2(i) = \frac{1}{L}\sum_{l=1}^{L}\exp\left(-\frac{e^2(f(l))}{2\sigma_m^2}\right), \tag{29}$$

where $e(f(l)) = d(f(l)) - \boldsymbol{\Omega}^T(i-1)\boldsymbol{\Phi}(\boldsymbol{x}(f(l)))$ and f is a set to store the indices of L randomly selected samples from $\{\boldsymbol{x}(j), d(j)\}_{j=1}^{i}$.

To obtain the update form of weight vector, we first define the following variables

$$\boldsymbol{d}_f(i) = \left[d(f(1)), \cdots, d(f(L))\right]^T, \tag{30}$$

$$\boldsymbol{\Phi}_f(i) = \left[\boldsymbol{\Phi}(\boldsymbol{x}(f(1))), \cdots, \boldsymbol{\Phi}(\boldsymbol{x}(f(L)))\right]. \tag{31}$$

Then, the prediction error $\boldsymbol{e}(i)$ can be obtained by

$$\boldsymbol{e}_f(i) = \boldsymbol{d}_f(i) - \boldsymbol{\Phi}_f^T(i)\boldsymbol{\Omega}(i-1). \tag{32}$$

The derivative of $J_2(i)$ defined in (29) versus the weights can be written as

$$\begin{aligned}\frac{\nabla J_2(i)}{\nabla \boldsymbol{\Omega}(i-1)} &= \frac{1}{\sigma_m^2 L}\sum_{l=1}^{L}\exp\left(\frac{-e^2(f(l))}{2\sigma_m^2}\right)e(f(l))\boldsymbol{\Phi}(\boldsymbol{x}(f(l))) \\ &= \frac{1}{\sigma_m^2 L}\boldsymbol{\Phi}_f(i)\boldsymbol{\varphi}_f^T(i),\end{aligned} \tag{33}$$

where $\boldsymbol{\varphi}_f(i) = \left[\varphi(f(1)), \varphi(f(2)), \cdots, \varphi(f(L))\right]$. On the basis of (33), the equation to update the weight vector of SB-GHFFMC algorithm can be given by

$$\boldsymbol{\Omega}(i) = \boldsymbol{\Omega}(i-1) + \frac{\mu}{L}\boldsymbol{\Phi}_f(i)\boldsymbol{\varphi}_f^T(i). \tag{34}$$

It is clear that SB-GHFFMC will reduce to the GHFFMC algorithm if we set $L = 1$ and $f(l) = i$, which means that SB-GHFFMC is actually a general extension to the latter and is thus expected to obtain better learning performance.

Finally, the proposed SB-GHFFMC and GHFFMC algorithms are summarized in Algorithm 2.

Algorithm 2 GHFFMC and SB-GHFFMC Algorithms

Input: Input-output data pairs $\{\boldsymbol{x}(i), d(i)\}$ for $i = 1, 2, \cdots$, extended dimension D, reuse length D_p.

Initialization: $\boldsymbol{\Omega}(1) = \boldsymbol{0}$, $\{b\} \sim U(0, 2\pi)$, calculate the Gaussian quadrature points $\{\widetilde{\boldsymbol{\omega}}_i\}_{i=1}^{D}$ and weight coefficients $\{\tilde{h}_i\}_{i=1}^{D}$ according to Algorithm 1 and (25).

Computation:

for $i = 2, 3, \cdots$ **do**

1. GHFFMC
   1) Compute $\boldsymbol{\Phi}(\boldsymbol{x}(i))$:
   $$\boldsymbol{\Phi}(\boldsymbol{x}(i)) = \begin{bmatrix}\sqrt{2\tilde{h}_1}\cos(\widetilde{\boldsymbol{\omega}}_1^T\frac{\sqrt{2}}{\sigma}\boldsymbol{x}(i) + b_1)\\ \vdots \\ \sqrt{2\tilde{h}_1}\cos(\widetilde{\boldsymbol{\omega}}_D^T\frac{\sqrt{2}}{\sigma}\boldsymbol{x}(i) + b_D)\end{bmatrix};$$
   2) Compute the prediction error:
   $e(i) = d(i) - \boldsymbol{\Omega}^T(i)\boldsymbol{\Phi}(\boldsymbol{x}(i))$;
   3) Update the weight vector:
   $\boldsymbol{\Omega}(i) = \boldsymbol{\Omega}(i-1) + \mu\varphi(i)\boldsymbol{\Phi}(\boldsymbol{x}(i))$.
2. SB-GHFFMC
   1) Get the set f that is used to store the indices of L randomly selected samples from $\{\boldsymbol{x}(j), d(j)\}_{j=1}^{i}$;
   2) Set $\boldsymbol{d}_f(i)$ and $\boldsymbol{\Phi}_f(i)$ following (30) and (31), respectively;
   3) Compute $\boldsymbol{e}_f(i)$ by $\boldsymbol{e}_f(i) = \boldsymbol{d}_f(i) - \boldsymbol{\Phi}_f^T(i)\boldsymbol{\Omega}(i-1)$;
   4) Update the weight vector:
   $\boldsymbol{\Omega}(i) = \boldsymbol{\Omega}(i-1) + \frac{\mu}{L}\boldsymbol{\Phi}_f(i)\boldsymbol{\varphi}_f^T(i)$.

end

IV. Theoretical Analysis

In this section, the stability and steady-state MSD performance of GHFFMC and SB-GHFFMC algorithms are analyzed. Without loss of generality, we assume that there exists an optimal weight vector $\boldsymbol{\Omega_o}$ such that

$$d(i) = \boldsymbol{\Omega_o}^T\boldsymbol{\Phi}(i) + v(i), \tag{35}$$

where $v(i)$ is the disturbance noise with the variance of σ_v^2. For convenience, $\boldsymbol{\Phi}(i)$ is used to denote $\boldsymbol{\Phi}(\boldsymbol{x}(i))$.

We subtract both sides of (34) separately from $\boldsymbol{\Omega_o}$ and obtain the expression of the weight error vector

$$\begin{aligned}\tilde{\boldsymbol{\Omega}}(i) &= \boldsymbol{\Omega_o} - \boldsymbol{\Omega}(i) \\ &= \tilde{\boldsymbol{\Omega}}(i-1) - \frac{\mu}{L}\boldsymbol{\Phi}_f(i)\boldsymbol{\varphi}_f^T(i).\end{aligned} \tag{36}$$

Taking the expectation of the square of the Euclidean norm of the weight error vector $\tilde{\boldsymbol{\Omega}}(i)$ to get the expression of MSD as

$$\mathrm{MSD}(i) = E\left[\left\|\tilde{\boldsymbol{\Omega}}(i)\right\|^2\right]. \tag{37}$$

Meanwhile, the following assumptions are provided for facilitating the analysis.

$A1$: $v(i)$ is zero-mean, independent, identically distributed, and is independent of transformed input signals $\boldsymbol{\Phi}(\boldsymbol{x})$ [42].

$A2$: The weight error vector $\tilde{\boldsymbol{\Omega}}$, the transformed input vector $\boldsymbol{\Phi}(\boldsymbol{x})$, and the noise $v(i)$ are statistically independent [43].

$A3$: The filter is long enough such that $\boldsymbol{\Phi}^T(i)\boldsymbol{\Phi}(i)$ and $\varphi^2(i)$ are uncorrelated [18]. Meanwhile, $E\left[\boldsymbol{\Phi}^T(i)\boldsymbol{\Phi}(i)\right] = D\sigma_{\boldsymbol{\Phi}}^2$, where $\sigma_{\boldsymbol{\Phi}}^2$ denotes the variance of $\boldsymbol{\Phi}(i)$.

A. Convergence Condition

To ensure the stability of the SB-GHFFMC algorithm, the energy of the weight error vector should decrease monotonically, that is, $\mathrm{MSD}(i) - \mathrm{MSD}(i-1) \le 0$.

Combining (36) and (37), we obtain

$$\begin{aligned}
&\mathrm{MSD}(i)\\
&= E\left[\left\|\tilde{\boldsymbol{\Omega}}(i-1)-\frac{\mu}{L}\boldsymbol{\Phi}_f(i)\boldsymbol{\varphi}_f^T(i)\right\|^2\right]\\
&= E\left[\left\|\tilde{\boldsymbol{\Omega}}(i-1)\right\|^2\right]+\frac{\mu^2}{L^2}E\left[\left\|\boldsymbol{\Phi}_f(i)\boldsymbol{\varphi}_f^T(i)\right\|^2\right]\\
&\quad-\frac{2\mu}{L}E\left[\boldsymbol{\varphi}_f(i)\boldsymbol{\Phi}_f^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right]\\
&= \mathrm{MSD}(i-1)\\
&\quad+\frac{\mu^2}{L^2}E\left[\sum_{l=1}^{L}\varphi(f(l))\boldsymbol{\Phi}^T(f(l))\boldsymbol{\Phi}(f(l))\varphi(f(l))\right]\\
&\quad-\frac{2\mu}{L}E\left[\sum_{l=1}^{L}\varphi(f(l))\boldsymbol{\Phi}^T(f(l))\tilde{\boldsymbol{\Omega}}(i-1)\right],
\end{aligned}\tag{38}$$

where $\boldsymbol{\Phi}(f(l)) = \boldsymbol{\Phi}(\boldsymbol{x}(f(l)))$. Applying (38) under the restriction $\mathrm{MSD}(i)-\mathrm{MSD}(i-1)\le 0$, we finally obtain the following condition to ensure the stability and convergence of the SB-GHFFMC algorithm

$$0<\mu\le\frac{2LE\left[\sum_{l=1}^{L}\varphi(f(l))\boldsymbol{\Phi}^T(f(l))\tilde{\boldsymbol{\Omega}}(i-1)\right]}{E\left[\sum_{l=1}^{L}\varphi(f(l))\boldsymbol{\Phi}^T(f(l))\boldsymbol{\Phi}(f(l))\varphi(f(l))\right]}.\tag{39}$$

Further, it should be noted that the proposed GHFFMC algorithm is essentially a special case of the SB-GHFFMC algorithm by setting $L=1$ and $f(i)=i$. Hence, (39) also offers a convergence condition for the GHFFMC algorithm with $L=1$ and $f(i)=i$.

B. Steady-State Mean-Square-Deviation

When the SB-GHFFMC algorithm reaches the steady state, it can be deduced that

$$E\left[e_a(f(l))\right]=E\left[e_a(i)\right],\quad \forall f(l)\tag{40}$$

$$E\left[e(f(l))\right]=E\left[e(i)\right],\quad \forall f(l),\tag{41}$$

where $e_a(i)$ is a prior error at the moment i and $e(i)$ is the corresponding prediction error. Given that $e_a(i)=\tilde{\boldsymbol{\Omega}}^T(i-1)\boldsymbol{\Phi}(i)$ and $e(i)=e_a(i)+v(i)$ for the SB-GHFFMC algorithm, we further obtain

$$\begin{aligned}
&E\left[\boldsymbol{\Phi}^T(f(l))\tilde{\boldsymbol{\Omega}}(i-1)\right]\\
&\qquad=E\left[\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right],\quad \forall f(l)
\end{aligned}\tag{42}$$

$$\begin{aligned}
&E\left[\boldsymbol{\Phi}^T(f(l))\tilde{\boldsymbol{\Omega}}(i-1)+v(f(l))\right]\\
&\qquad=E\left[\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)+v(i)\right].\quad \forall f(l)
\end{aligned}\tag{43}$$

Combining (38), (42) and (43), we have

$$\begin{aligned}
\mathrm{MSD}(i)&\approx\mathrm{MSD}(i-1)+\frac{\mu^2}{L}E\left[\varphi^2(i)\|\boldsymbol{\Phi}(i)\|^2\right]\\
&\quad-2\mu E\left[\varphi(i)\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right]\\
&=\mathrm{MSD}(i-1)+\frac{\mu^2}{L}E\left[\varphi^2(i)\right]Tr\left[\boldsymbol{\Phi}(i)\boldsymbol{\Phi}^T(i)\right]\\
&\quad-2\mu E\left[\varphi(i)\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right],
\end{aligned}\tag{44}$$

where $Tr[\cdot]$ denotes the trace of a matrix.

Taking the Taylor series expansion of $\varphi(i)$ at $v(i)$ yields

$$\begin{aligned}
\varphi(i)&=\varphi\left(\tilde{\boldsymbol{\Omega}}^T(i-1)\boldsymbol{\Phi}(i)+v(i)\right)\\
&=\varphi_v+\varphi'_v\tilde{\boldsymbol{\Omega}}^T(i-1)\boldsymbol{\Phi}(i)\\
&\quad+\frac{1}{2}\varphi''_v(\tilde{\boldsymbol{\Omega}}^T(i-1)\boldsymbol{\Phi}(i))^2+o,
\end{aligned}\tag{45}$$

where $\varphi_v=\exp\left(-\frac{v^2(i)}{2\sigma_m^2}\right)v(i)$, φ'_v and φ''_v are the first-order and second-order derivatives of φ_v with respect to $v(i)$, respectively, and o denotes the high-order terms of $\tilde{\boldsymbol{\Omega}}^T(i-1)\boldsymbol{\Phi}(i)$ and is always assumed to be small enough and ignorable in the following.

On the basis of (45), $E\left[\varphi^2(i)\right]$ in (44) can be written as

$$\begin{aligned}
&E\left[\varphi^2(i)\right]\\
&\approx E\left[\varphi_v^2\right]+E\left[\varphi_v\varphi''_v+(\varphi'_v)^2\right]E\left[(\tilde{\boldsymbol{\Omega}}^T(i-1)\boldsymbol{\Phi}(i))^2\right]\\
&=E\left[\varphi_v^2\right]+E\left[\varphi_v\varphi''_v+(\varphi'_v)^2\right]\sigma_{\boldsymbol{\Phi}}^2\mathrm{MSD}(i-1).
\end{aligned}\tag{46}$$

Similarly, the last term in (44) can be written as

$$\begin{aligned}
&E\left[\varphi(i)\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right]\\
&\approx E\left[\varphi_v\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right]+E\left[\varphi'_v\left(\boldsymbol{\Phi}^T(i)\tilde{\boldsymbol{\Omega}}(i-1)\right)^2\right]\\
&=E\left[\varphi'_v\right]\sigma_{\boldsymbol{\Phi}}^2MSD(i-1).
\end{aligned}\tag{47}$$

It worth noting that, according to Assumption $A3$, the following equation holds

$$\sigma_{\boldsymbol{\Phi}}^2=\frac{Tr\left[\boldsymbol{\Phi}(i)\boldsymbol{\Phi}^T(i)\right]}{D}.\tag{48}$$

Hence, the expression for the steady-state MSD of the SB-GHFFMC algorithm can be obtained by combining (44), (46), (48), and (47), i.e.,

$$\begin{aligned}
&\mathrm{MSD}(\infty)\\
&=\frac{D\mu E\left[\varphi_v^2\right]}{2LE[\varphi'_v]-\mu Tr\left[\boldsymbol{\Phi}(i)\boldsymbol{\Phi}^T(i)\right]E\left[\varphi_v\varphi''_v+(\varphi'_v)^2\right]}.
\end{aligned}\tag{49}$$

It is evident that the theoretical value of the steady-state MSD of the SB-GHFFMC algorithm depends on $E\left[\varphi_v^2\right]$, $E\left[\varphi'_v\right]$, and $E\left[\varphi_v\varphi''_v+(\varphi'_v)^2\right]$. Once the probability density function (PDF) for the noise v is available, the values of these three expectations can be obtained. For instance, if the disturbance noise $v(i)$ is assumed to follow a Gaussian distribution with zero-mean and variance σ_v^2, the value of $E\left[\varphi_v^2\right]$ in (49) can be calculated by

$$\begin{aligned}
E\left[\varphi_v^2\right]&=E\left[\exp\left(-\frac{v^2(i)}{\sigma_m^2}\right)v^2(i)\right]\\
&=\int_{-\infty}^{+\infty}p(v)\exp\left(-\frac{v^2(i)}{\sigma_m^2}\right)v^2(i)\,dv(i)\\
&=\int_{-\infty}^{+\infty}\frac{1}{\sqrt{2\pi}\sigma_v}\exp\left(-\left(\frac{1}{2\sigma_v^2}+\frac{1}{\sigma_m^2}\right)v^2(i)\right)v^2(i)\,dv(i)\\
&=\frac{\sigma_v^2\sigma_m^3}{\left(\sigma_m^2+2\sigma_v^2\right)^{\frac{3}{2}}}.
\end{aligned}\tag{50}$$

where $p(v)$ denotes the probability density function of $v(i)$. Meanwhile, the values of $E\left[\varphi'_v\right]$ and $E\left[\varphi_v\varphi''_v+\left(\varphi'_v\right)^2\right]$ in (49) can be, respectively, given by

$$\begin{aligned}E\left[\varphi'_v\right]&=E\left[\exp\left(-\frac{v^2(i)}{2\sigma_m^2}\right)\left(1-\frac{v^2(i)}{\sigma_m^2}\right)\right]\\&=\int_{-\infty}^{+\infty}p(v)\exp\left(-\frac{v^2(i)}{2\sigma_m^2}\right)\left(1-\frac{v^2(i)}{\sigma_m^2}\right)dv(i)\\&=\frac{\sigma_m}{\left(\sigma_m^2+\sigma_v^2\right)^{\frac{1}{2}}}-\frac{\sigma_v^2\sigma_m}{\left(\sigma_m^2+\sigma_v^2\right)^{\frac{3}{2}}},\end{aligned}\tag{51}$$

and

$$\begin{aligned}&E\left[\varphi_v\varphi''_v+\left(\varphi'_v\right)^2\right]\\&=E\left[\exp\left(-\frac{v^2(i)}{\sigma_m^2}\right)\left(1-\frac{5v^2(i)}{\sigma_m^2}+\frac{2v^4(i)}{\sigma_m^4}\right)\right]\\&=\int_{-\infty}^{+\infty}p(v)\exp\left(-\frac{v^2(i)}{\sigma_m^2}\right)\\&\quad\times\left(1-\frac{5v^2(i)}{\sigma_m^2}+\frac{2v^4(i)}{\sigma_m^4}\right)dv(i)\\&=\frac{\sigma_m}{\left(2\sigma_v^2+\sigma_m^2\right)^{\frac{1}{2}}}-\frac{2\sigma_v^2}{\sigma_m\left(2\sigma_v^2+\sigma_m^2\right)^{\frac{3}{2}}}.\end{aligned}\tag{52}$$

Substituting (50), (51), and (52) into (49), the final theoretical value of steady-state MSD for the SB-GHFFMC algorithm in a Gaussian noise can be obtained by

$$\mathrm{MSD}(\infty)=\frac{D\mu\sigma_v^2\sigma_m^3}{2L\left(\sigma_m^2+\frac{\sigma_m^2\sigma_v^2}{\sigma_m^2+\sigma_v^2}\right)^{\frac{3}{2}}-\mu Tr\left[\boldsymbol{\Phi}(i)\boldsymbol{\Phi}^T(i)\right]\zeta},\tag{53}$$

where $\zeta=\left(2\sigma_v^2\sigma_m^2+\sigma_m^4-2\sigma_v^2\right)$.

Although the aforementioned method provides a way to accurately calculate the value of the steady-state MSD shown in (49), it involves cumbersome calculations of numerical integrations for various noise distributions. For example, if the disturbance noise $v(i)$ is assumed to follow a uniform distribution, we need to accurately recalculate the values of $E\left[\varphi_v^2\right]$, $E\left[\varphi'_v\right]$, and $E\left[\varphi_v\varphi''_v+\left(\varphi'_v\right)^2\right]$, which is a very complex process for some special noise distributions. To offer a simple and efficient way to calculate the value of the steady-state MSD for the SB-GHFFMC algorithm, the sample estimator method [44] will be adopted to approximate the expectation. With a sufficiently large sample number, this method provides a reliable estimate. For example, once the PDF for the noise v is available, we can generate numerous noise samples $\{v(n)\}_{n=1}^N$ based on this given distribution, where N denotes the number of samples. Therefore, the values of $\{\varphi_v(n)\}_{n=1}^N$, $\{\varphi_v^2(n)\}_{n=1}^N$, $\{\varphi'_v(n)\}_{n=1}^N$, and $\{\varphi''_v(n)\}_{n=1}^N$ can be obtained. Based on these results, we have

$$\begin{aligned}E\left[\varphi_v^2\right]&=\frac{1}{N}\sum_{n=1}^N\varphi_v^2(n)\\&=\frac{1}{N}\sum_{n=1}^N\exp\left(-\frac{v^2(n)}{\sigma_m^2}\right)v^2(n),\end{aligned}\tag{54}$$

$$\begin{aligned}E\left[\varphi'_v\right]&=\frac{1}{N}\sum_{n=1}^N\varphi'_v(n)\\&=\frac{1}{N}\sum_{n=1}^N\exp\left(-\frac{v^2(n)}{2\sigma_m^2}\right)\left(1-\frac{v^2(n)}{\sigma_m^2}\right),\end{aligned}\tag{55}$$

and

$$\begin{aligned}&E\left[\varphi_v\varphi''_v+\left(\varphi'_v\right)^2\right]\\&=\frac{1}{N}\sum_{n=1}^N\left[\varphi_v(n)\varphi''_v(n)+\left(\varphi'_v(n)\right)^2\right]\\&=\frac{1}{N}\sum_{n=1}^N\exp\left(-\frac{v^2(n)}{\sigma_m^2}\right)\left(1-\frac{5v^2(n)}{\sigma_m^2}+\frac{2v^4(n)}{\sigma_m^4}\right).\end{aligned}\tag{56}$$

Substituting (54), (55), and (56) into (49), we can obtain the final theoretical expression of the steady-state MSD. It is clear that the used sample estimator method is suitable for various noise distributions without cumbersome calculations of numerical integrations, and is thus a relatively simple and straightforward method to calculate the value of the steady-state MSD for the SB-GHFFMC algorithm.

Remark 2: Although the theoretical analysis framework adopted in this section is somewhat similar to that of [18], there are two main differences between them. Firstly, the theoretical analysis presented in [18] focuses on the steady-state excess mean-square error (EMSE), which is another metric that can reflect the accuracy of predicted outputs but cannot directly assess the accuracy of the estimated filtering weights or coefficients. On the contrary, the theoretical analysis presented in this section focuses on the steady-state MSD, which is an intuitive measure of the accuracy of the estimated filtering weights or coefficients. Secondly, the calculation of the final steady-state EMSE in [18] heavily relies on a strict numerical integration method, which may involve a very complex computation process for some special noise distributions. Conversely, the sample estimator method introduced in this section provides a simple and straightforward way to compute the final steady-state MSD.

C. Computational Complexity Analysis

In this section, the computational complexities of the proposed two algorithms at each iteration are analyzed and provided in Table I, where the nonlinear operations mainly contain $\cos(\cdot)$ and $\exp(\cdot)$. For comparison, the computational complexities of KLMS [9], KMC [31], RFFKLMS [4], RB-RFFMC [18], and RFFMC [18] are chosen. Here, i stands for the number of iteration steps, N stands for the dimension of the input data, D stands for the dimension of the data after RFF and GHFF transformations, and L stands for the length of the stochastic batch in the RB-RFFMC and SB-GHFFMC algorithms. Meanwhile, we always assume that the RFF or GHFF mapping has already been completed before analyzing the computational complexity of a specific algorithm.

It can be observed from Table I that, the computational complexity of the GHFFMC algorithm is the same as that of the RFFMC algorithm and significantly lower than that of the KMC algorithm. Meanwhile, the computational complexities

TABLE I
COMPUTATIONAL COMPLEXITIES OF DIFFERENT ALGORITHMS PER ITERATION

Algorithm	Addition	Multiplication	Nonlinear
KLMS [9]	$2Ni-2N+2$	$Ni-N+5i-4$	$i-1$
KMC [31]	$2Ni-2N+2$	$Ni-N+5i+1$	i
RFFKLMS [4]	$ND+2D$	$ND+3D+2$	$D+1$
RFFMC [18]	$ND+2D$	$ND+3D+7$	$D+2$
RB-RFFMC [18]	$ND+2DL$	$ND+2DL+2D+5L+2$	$D+L+1$
GHFFMC	$ND+2D$	$ND+3D+7$	$D+2$
SB-GHFFMC	$ND+2DL$	$ND+2DL+2D+5L+2$	$D+L+1$

of the KLMS and KMC algorithms are proportional to i, whereas the complexity of the SB-GHFFMC algorithm is only related to the data dimension. Therefore, when i increases, the SB-GHFFMC algorithm can reduce the computational complexity of the corresponding KAF, significantly.

V. SIMULATIONS

In this section, we conduct simulations to validate the theoretical analysis and demonstrate the desired performance of the two proposed algorithms in the scenarios of nonlinear system identification and prediction of Chua's circuit time-series [45].

A. Theoretical Verification

To validate the theoretical analysis, we generate 10000 input-output data pairs $\{\boldsymbol{x}(i), d(i)\}_{i=1}^{N}$, where $\{\boldsymbol{x}(i)\}_{i=1}^{10000}$ are the input signals generated through a standard Gaussian distribution, and $\{d(i)\}_{i=1}^{10000}$ are the corresponding outputs generated by $d(i) = \boldsymbol{\Omega_o}^T \boldsymbol{\Phi}(\boldsymbol{x}(i)) + v(i), i = 1, \cdots, 10000$. Except mentioned otherwise, the values of D and D_p are set to 5 to construct $\{\boldsymbol{\Phi}(\boldsymbol{x}(i))\}_{i=1}^{N}$ through GHFF mapping method. Meanwhile, the optimal weight vector $\boldsymbol{\Omega_o}$ is randomly selected from the transformed input signal and normalized by $\boldsymbol{\Omega}_o^T \boldsymbol{\Omega}_o = 1$.

1) *Verification of the Convergence Condition:* To validate the efficiency of the convergence condition presented in (39) (with L set to 1), we show the square of the error of the GHFFMC algorithm with the maximum step size μ_m and other step sizes in Fig. 4. Meanwhile, μ_m denotes the upper bound of the step size for convergence, and the disturbance noise is set to follow an α-stable distribution [18] with the model parameters $V = (2, 0, 0.3, 0)$ to model the influence of outliers. As can be observed from Fig. 4, the GHFFMC algorithm fails to converge when the step size exceeds the upper bound μ_m. However, when the step size is smaller than the upper bound, the GHFFMC algorithm converges as expected. Thus, the validity of the convergence condition is verified.

2) *Verification of the Theoretical Steady-state MSD:* For further verifying the theoretical steady-state MSD presented in (49), Fig. 5 shows its values and simulated ones in a Gaussian noise environment. Here, GHFFMC-T and SB-GHFFMC-T represent the theoretical steady-state MSDs for GHFFMC and SB-GHFFMC algorithms, while GHFFMC-S and SB-GHFFMC-S represent the simulated steady-state MSDs of GHFFMC and SB-GHFFMC, respectively. Additionally, the simulated value of steady-state MSD is obtained as an average over the last 100 iterations of MSD curves. To reduce random error caused by simulations, all the obtained results have been averaged over 100 independent Monte Carlo runs.

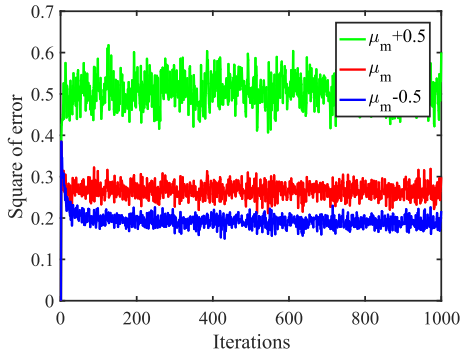


Fig. 4. Learning curves of the GHFFMC algorithm under different step sizes.

From Fig. 5, the following observations can be obtained:

- The theoretical steady-state MSDs of the GHFFMC and SB-GHFFMC algorithms match well with their simulated ones, which confirms the correctness of the results of the theoretical analysis.
- When other parameters are fixed and only the value of the step size μ or the kernel width σ_m in correntropy is adjusted, a larger μ or σ_m can result in a larger steady-state MSD. This means that both μ and σ_m are two crucial parameters to enhance the desired performance of the proposed algorithms, and their values should be adjusted appropriately.
- The steady-state MSD of SB-GHFFMC decreases as L increases. However, when L increases to 16, the reduced value of the steady-state MSD is limited. To balance the computational complexity and filtering accuracy, we set $L = 16$ as the default choice in the following.

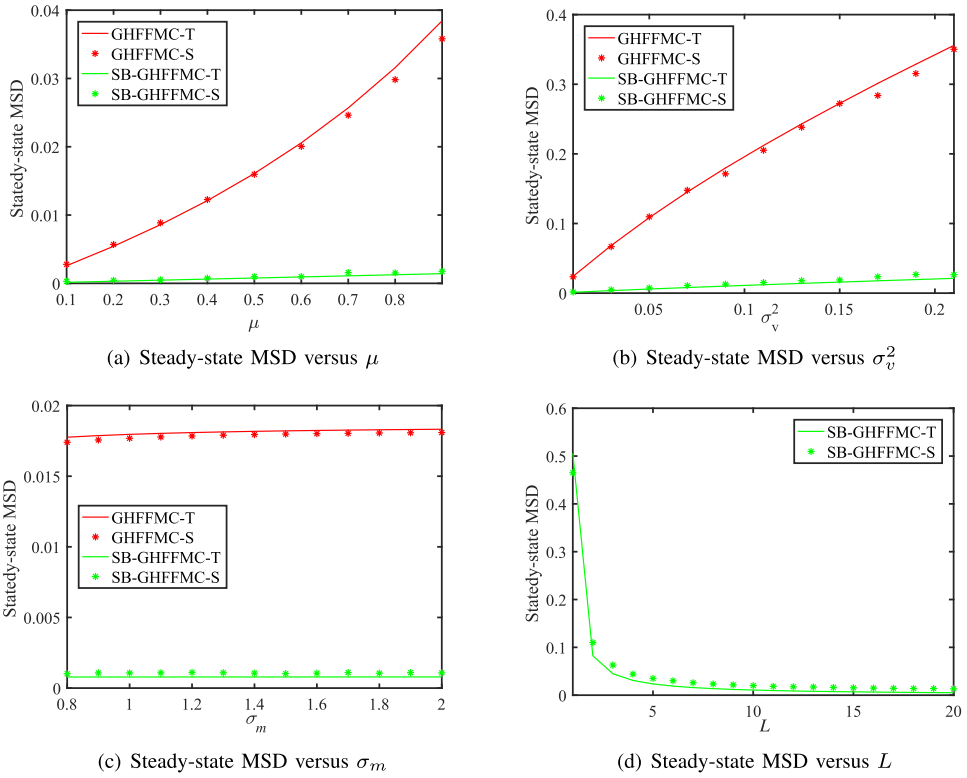
Additionally, we choose Laplace noise with probability density function $p(v) = \frac{1}{2\alpha}\exp\left(-\frac{|v|}{\alpha}\right)$ as the non-Gaussian disturbance noise and verify the theoretical analysis under this case. The related results can be found in Fig. 6, where σ_v^2 denotes the variance of Laplace noise and is calculated by $\sigma_v^2 = 2\alpha^2$. It can be seen from Fig. 6 that, the theoretical steady-state MSDs of the GHFFMC and SB-GHFFMC algorithms still closely align with their simulated ones, thereby further confirming the correctness of the theoretical analysis.

B. Nonlinear System Identification

We now test the performance of the proposed algorithms in the scenario of nonlinear system identification. Similar to [46], the model of the system is described as

$$\begin{aligned} x(n) = &\left(\varepsilon_1 - \varepsilon_2 \exp\left(-x^2(n-1)\right)\right) x(n-1) \\ &- \left(\varepsilon_3 + \varepsilon_4 \exp\left(-x^2(n-1)\right)\right) x(n-2) \\ &+ \varepsilon_5 \sin(\pi x(n-1)), \end{aligned} \tag{57}$$

where $x(n)$ denotes the output of the nth sample, obtained from the prediction of $[x(n-2), x(n-1)]^T$ with



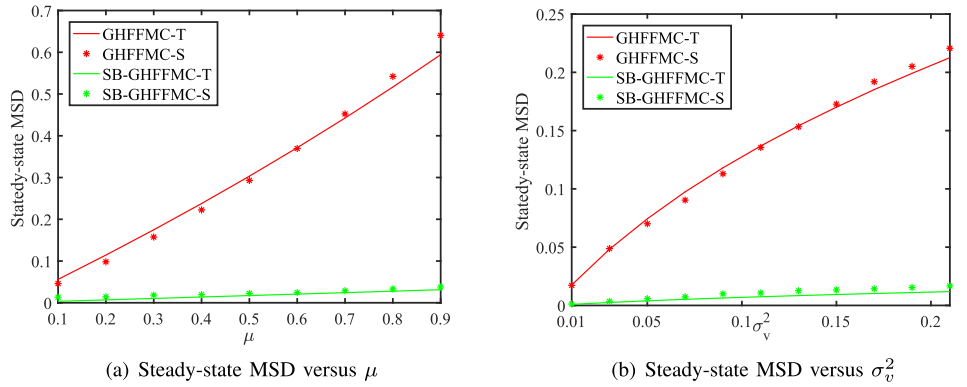
(a) Steady-state MSD versus μ

(b) Steady-state MSD versus σ_v^2

(c) Steady-state MSD versus σ_m

(d) Steady-state MSD versus L

Fig. 5. Comparison of theoretical and simulated MSDs in the presence of Gaussian noise. (a) $\sigma_v^2 = 0.01, \sigma_m = 0.8, L = 16$; (b) $\mu = 0.8, \sigma_m = 0.8, L = 16$; (c) $\mu = 0.5, \sigma_v^2 = 0.01, L = 16$; (d) $\mu = 4, \sigma_v^2 = 0.01, \sigma_m = 0.7$.



(a) Steady-state MSD versus μ

(b) Steady-state MSD versus σ_v^2

Fig. 6. Comparison of theoretical and simulated MSDs in the presence of non-Gaussian noise. (a) $\sigma_v^2 = 0.5, \sigma_m = 1, L = 16$; (b) $\mu = 0.6, \sigma_m = 1, L = 16$.

$x(1) = x(2) = 0.1$. Meanwhile, $\boldsymbol{\varepsilon} = [\varepsilon_1, \varepsilon_2, \varepsilon_3, \varepsilon_4, \varepsilon_5]$ denotes the vector of coefficients, which is directly set to $[0.8, 0.5, 0.3, 0.9, 0.1]$. We run 100 independent Monte Carlo simulations under Gaussian and non-Gaussian disturbance noise environments, respectively. To make a comprehensive comparison, the results obtained with KLMS, KMC, RFFLMS, RFFMC, and RB-RFFMC are also provided. In each run, 1000 samples are used as training data and 100 samples are used as test data. Since the optimal weight vector is unavailable in this example, the commonly used testing mean square error (MSE) [47] is therefore employed as the default measure criterion to assess the filtering accuracy, i.e.,

$$\text{MSE} = \frac{1}{M}\sum_{k=1}^{M} e_t^2(k), \tag{58}$$

where $e_t(k)$ denotes the estimated error of the kth test sample and M denotes the total number of test samples.

1) *Gaussian Noise Environment:* By setting the disturbance noise to be drawn from a Gaussian distribution with zero-mean and variance $\sigma_v^2 = 0.2$, Fig 7 shows the testing MSEs for different algorithms. Here, we set $D = 100$ to perform RFF or GHFF mapping, $D_p = 10$ for GHFF mapping, and $\sigma = 2$ to perform the standard kernel mapping. The parameter σ_m to design the correntropy loss is set to 4, and the parameter L in SB-GHFFMC and RB-RFFMC is set to 16. Meanwhile, the step sizes for SB-GHFFMC, GHFFMC, RB-RFFMC, RFFMC, RFFKLMS, KLMS, and KMC are set to 0.7, 0,3, 0.9, 0.4, 0.4, 0.12, and 0.12, respectively, to ensure that these algorithms have almost the same initial convergence speed. As can be seen from Fig. 7, although the final value of the testing MSE for GHFFMC is slightly

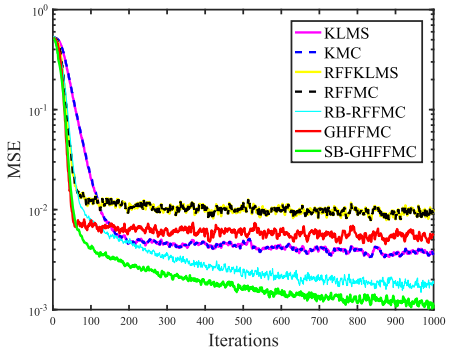


Fig. 7. Learning curves of different algorithms for system identification in the presence of Gaussian noise.

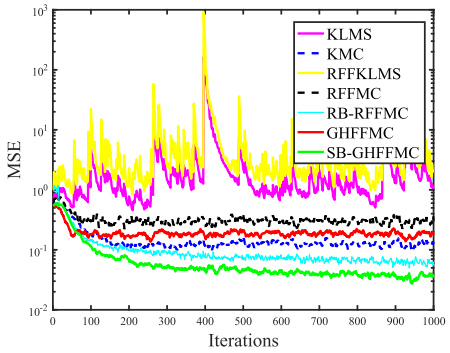


Fig. 8. Learning curves of different algorithms for system identification in the presence of non-Gaussian noise.

larger than those of KLMS and KMC, it is smaller than those of RFFMC and RFFKLMS. This suggests that an adaptive learning algorithm designed with the GHFF method can be a better candidate to perform the task of nonlinear system identification in comparison with the one designed with the RFF method. Moreover, SB-GHFFMC can obtain a smaller testing MSE than GHFFMC and RB-RFFMC, which confirms the effectiveness of using the historical information of the data to enhance the filtering performance.

2) *Non-Gaussian Noise Environment:* To further test the performance of the proposed algorithms in non-Gaussian noise environments, we set the noise to be drawn from an α-stable distribution and the related model parameters are set as $V = (1.5, 0, 1, 0)$. Then, the learning curves for SB-GHFFMC, GHFFMC, RFFMC, RB-RFFMC, RFFKLMS, KLMS, and KMC can be found in Fig. 8. It can be seen from Fig. 8 that, the proposed GHFFMC algorithm can not only obtain a smaller testing MSE at the last iteration than KLMS, RFFKLMS, and RFFMC, but also can obtain a comparable performance to KMC. Meanwhile, the SB-GHFFMC algorithm has a lower final testing MSE compared to the GHFFMC and RB-RFFMC algorithms. The results suggest that the SB-GHFFMC and GHFFMC algorithms can still be two excellent candidates to perform the task of nonlinear system identification in the α-stable noise environment.

C. Prediction of Chua's Circuit Time-Series

Chua's circuits can induce complex nonlinear phenomena through the mathematical modelling of three-bit autonomous

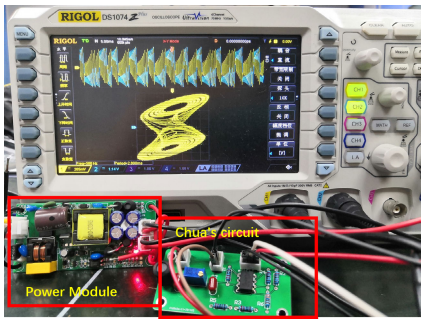


Fig. 9. Chua's circuit system [48].

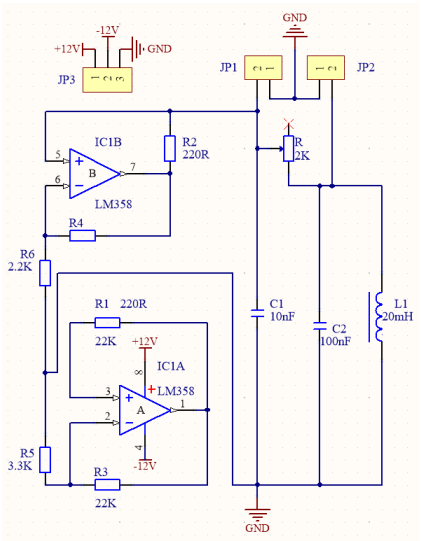
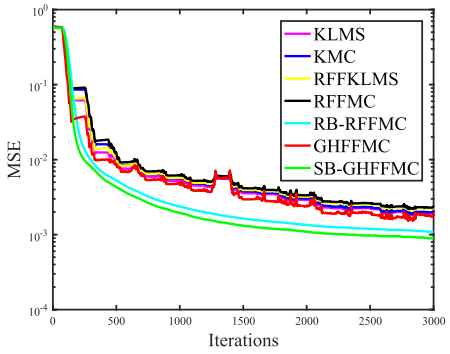


Fig. 10. Chua's circuit system schematic diagram [48].

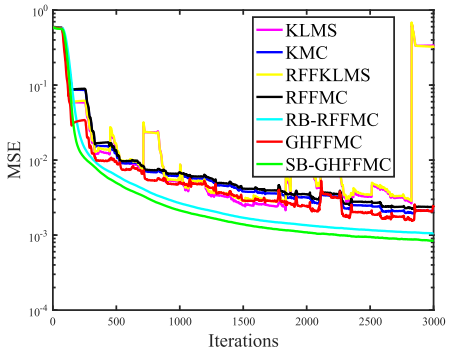
dynamical systems. In this section, a real Chua's circuit system is constructed to generate real-world chaotic time series, which are then used to test the performance of the proposed algorithms. The system of Chua's circuit is described as follows:

$$\begin{cases} \dot{u}_1 = \dfrac{u_2}{RC_1} - \dfrac{u_1}{RC_1} - \dfrac{g(u_1)}{C_1} \\ \dot{u}_2 = \dfrac{u_1}{RC_2} - \dfrac{u_2}{RC_2} - \dfrac{I_{\tilde{L}}}{C_2} \\ \dot{I}_{\tilde{L}} = -\dfrac{u_2}{\tilde{L}}, \end{cases} \tag{59}$$

where u_1, u_2, and $I_{\tilde{L}}$ represent the voltage of capacitors C_1, C_2, and the current of inductor $\tilde{L}$, respectively, and $g(u_1)$ is a segmented continuous function describing the diode voltage-ampere characteristic. Meanwhile, the system and schematic diagram of Chua's circuit are shown in Figs. 9 and 10 [48], respectively. In the following, we select the data of 5000 voltage values between C_1 and C_2 capacitors as the Chua's time-series. The past 5 latest voltage values at capacitor C_1 and C_2 are used as inputs to predict the current value at inductor $\tilde{L}$. The first 3000 noise contaminated samples are used to construct a training set and another 100 samples are used to construct a testing set to test the performance of different algorithms. Here, the testing MSE defined in (58) is also used as a performance measure criterion due to the

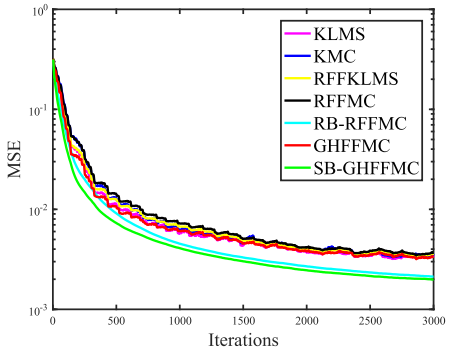


(a) Voltage data at C1

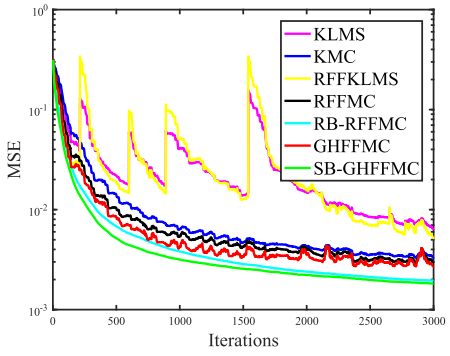


(b) Voltage data at C2

Fig. 11. Learning curves of different algorithms for prediction of Chua's circuit time-series in the presence of Gaussian noise.



(a) Voltage data at C1



(b) Voltage data at C2

Fig. 12. Learning curves of different algorithms for prediction of Chua's circuit time-series in the presence of non-Gaussian noise.

unavailable optimal weight vector in prediction of Chua's time-series.

Figs. 11 and 12 show the learning curves of different algorithms under Gaussian and non-Gaussian noise environments, respectively. Here, we configure the Gaussian noise with zero-mean and variance of 0.1, and configure the non-Gaussian noise with an α-stable distribution by setting $V = [0.9, 0, 0.01, 0]$. To make a fair comparison, the parameters for all compared methods have been appropriately chosen, similar to Section V-B. In particular, the parameter D in SB-GHFFMC, GHFFMC, RB-RFFMC, RFFMC, and RFFKLMS is set to 100, the parameter D_p in SB-GHFFMC and GHFFMC is set to 10, the parameter σ_m in SB-GHFFMC, GHFFMC, RB-RFFMC, RFFMC, and KMC is set to 1, the parameter σ is set to 0.4, and the parameter L in SB-GHFFMC and RB-RFFMC is set to 16. Meanwhile, the step sizes for SB-GHFFMC, GHFFMC, RB-RFFMC, RFFMC, RFFKLMS, KMC, and KLMS are set to 0.06, 0,03, 0.06, 0.03, 0.02, 0.02, and 0.02, respectively.

It can be seen from Fig. 11 and Fig. 12 that, GHFFMC and SB-GHFFMC are still very competitive compared to other algorithms, which means that they can be good candidates to predict the Chua's circuit time-series even in the presence of Gaussian or non-Gaussian noise. Meanwhile, SB-GHFFMC can obtain smaller testing MSE than GHFFMC due to the use of stochastic batch optimization method, which is in consistent with our expectations.

VI. Conclusion

To tackle the challenge of expanding network size in KAFs, a GHFF mapping method was proposed in this paper to approximate a user-specified kernel function. On the basis of GHFF, two new adaptive learning algorithms called GHFF maximum correntropy (GHFFMC) and stochastic batch GHFF (SB-GHFFMC) are subsequently developed. The proposed algorithms provide lower computational complexity and similar filtering accuracy to the KAFs algorithm. The steady-state mean-square-deviation (MSD) of the proposed algorithm under various noise cases are theoretically analyzed and verified by simulations in system identification applications. Simulation results on the tasks of nonlinear system identification and Chua's circuit time-series prediction demonstrate that, the proposed two algorithms with limited feature dimensions can outperform other competitive adaptive filtering algorithms in both Gaussian and non-Gaussian noise environments. The main limitation of the proposed algorithms is that their performance is influenced by multiple free parameters. Hence, it would be of great interest to develop an adaptive strategy to select parameters in the future. Additionally, extending the GHFF method with other more advanced technologies to handle very high-dimensional data is also worth to be studied.

References

[1] H. Wang, K. Xu, P. X. Liu, and J. Qiao, "Adaptive fuzzy fast finite-time dynamic surface tracking control for nonlinear systems," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 10, pp. 4337–4348, Oct. 2021.

[2] K. Tian, C. Grebogi, and H. Ren, "Chaos generation with impulse control: Application to non-chaotic systems and circuit design," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 7, pp. 3012–3022, Jul. 2021.

[3] W. Liu, J. C. Príncipe, and S. Haykin, *Kernel Adaptive Filtering: A Comprehensive Introduction*. Hoboken, NJ, USA: Wiley, 2010.
[4] A. Singh, N. Ahuja, and P. Moulin, "Online learning with kernels: Overcoming the growing sum problem," in *Proc. IEEE Int. Workshop Mach. Learn. Signal Process. (MLSP)*, Sep. 2012, pp. 1–6.
[5] L. F. d. S. Coelho, L. Lovisolo, and M. P. Tcheou, "Adaptive filtering with reduced computational complexity using SOPOT arithmetic," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 746–756, Feb. 2022.
[6] L. Shi, J. Tan, J. Wang, Q. Li, L. Lu, and B. Chen, "Robust kernel adaptive filtering for nonlinear time series prediction," *Signal Process.*, vol. 210, Sep. 2023, Art. no. 109090.
[7] H. Zhao and B. Chen, *Efficient Nonlinear Adaptive Filters: Design, Analysis and Applications*. Cham, Switzerland: Springer, 2023.
[8] J.-W. Xu, A. R. C. Paiva, I. Park, and J. C. Príncipe, "A reproducing kernel Hilbert space framework for information-theoretic learning," *IEEE Trans. Signal Process.*, vol. 56, no. 12, pp. 5891–5902, Dec. 2008.
[9] W. Liu, P. P. Pokharel, and J. C. Principe, "The kernel least-mean-square algorithm," *IEEE Trans. Signal Process.*, vol. 56, no. 2, pp. 543–554, Feb. 2008.
[10] P. K. Meher and S. Y. Park, "Critical-path analysis and low-complexity implementation of the LMS adaptive algorithm," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 61, no. 3, pp. 778–788, Mar. 2014.
[11] V. C. Gogineni, R. Sambangi, D. Alex, S. Mula, and S. Werner, "Algorithm and architecture design of random Fourier features-based kernel adaptive filters," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 2, pp. 833–845, Feb. 2023.
[12] W. Liu, I. Park, and J. C. Príncipe, "An information theoretic approach of designing sparse kernel adaptive filters," *IEEE Trans. Neural Netw.*, vol. 20, no. 12, pp. 1950–1961, Dec. 2009.
[13] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 20, Dec. 2007, pp. 1177–1184.
[14] Y. Zheng, S. Wang, and B. Chen, "Identification of Hammerstein systems with random Fourier features and kernel risk sensitive loss," *Neural Process. Lett.*, vol. 55, no. 7, pp. 9041–9063, Feb. 2023.
[15] F. Liu, X. Huang, Y. Chen, and J. A. K. Suykens, "Random features for kernel approximation: A survey on algorithms, theory, and beyond," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 7128–7148, Oct. 2022.
[16] K. Xiong, H. H. C. Iu, and S. Wang, "Kernel correntropy conjugate gradient algorithms based on half-quadratic optimization," *IEEE Trans. Cybern.*, vol. 51, no. 11, pp. 5497–5510, Nov. 2021.
[17] K. Xiong, T. Zhang, G. Cui, S. Wang, and L. Kong, "Coalition game of radar network for multitarget tracking via model-based multiagent reinforcement learning," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 3, pp. 2123–2140, Jun. 2023.
[18] S. Wang et al., "Random Fourier filters under maximum correntropy criterion," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 10, pp. 3390–3403, Oct. 2018.
[19] T. Dao, C. M. De Sa, and C. Ré, "Gaussian quadrature for kernel features," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, Dec. 2017, pp. 6109–6119.
[20] A. Townsend, T. Trogdon, and S. Olver, "Fast computation of Gauss quadrature nodes and weights on the whole real line," *IMA J. Numer. Anal.*, vol. 36, no. 1, pp. 337–358, Jan. 2016.
[21] M. Mutny and A. Krause, "Efficient high dimensional Bayesian optimization with additivity and quadrature Fourier features," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Dec. 2018, pp. 9019–9030.
[22] J. He, G. Wang, K. Cao, H. Diao, G. Wang, and B. Peng, "Generalized minimum error entropy for robust learning," *Pattern Recognit.*, vol. 135, Mar. 2023, Art. no. 109188.
[23] J. C. Príncipe, *Information Theoretic Learning: Renyi's Entropy and Kernel Perspectives*. New York, NY, USA: Springer, 2010.
[24] Z. Li, L. Xing, and B. Chen, "Adaptive filtering with quantized minimum error entropy criterion," *Signal Process.*, vol. 172, Jul. 2020, Art. no. 107534.
[25] B. Chen, L. Xing, J. Liang, N. Zheng, and J. C. Príncipe, "Steady-state mean-square error analysis for adaptive filtering under the maximum correntropy criterion," *IEEE Signal Process. Lett.*, vol. 21, no. 7, pp. 880–884, Jul. 2014.
[26] L. Dang, B. Chen, S. Wang, Y. Gu, and J. C. Príncipe, "Kernel Kalman filtering with conditional embedding and maximum correntropy criterion," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 66, no. 11, pp. 4265–4277, Nov. 2019.
[27] L. Shi, L. Shen, and B. Chen, "An efficient parameter optimization of maximum correntropy criterion," *IEEE Signal Process. Lett.*, vol. 30, pp. 538–542, Jun. 2023.
[28] Y. Zheng, B. Chen, S. Wang, and W. Wang, "Broad learning system based on maximum correntropy criterion," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 7, pp. 3083–3097, Jul. 2021.
[29] A. Khalili, A. Rastegarnia, M. K. Islam, and T. Y. Rezaii, "Steady-state tracking analysis of adaptive filter with maximum correntropy criterion," *Circuits, Syst., Signal Process.*, vol. 36, no. 4, pp. 1725–1734, Apr. 2017.
[30] B. Chen, L. Xing, H. Zhao, N. Zheng, and J. C. Principe, "Generalized correntropy for robust adaptive filtering," *IEEE Trans. Signal Process.*, vol. 64, no. 13, pp. 3376–3387, Jul. 2016.
[31] S. Zhao, B. Chen, and J. C. Príncipe, "Kernel adaptive filtering with maximum correntropy criterion," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2011, pp. 2012–2017.
[32] S. Khirirat, H. R. Feyzmahdavian, and M. Johansson, "Mini-batch gradient descent: Faster convergence under data sparsity," in *Proc. IEEE 56th Annu. Conf. Decis. Control (CDC)*, Dec. 2017, pp. 2880–2887.
[33] A. Cotter, O. Shamir, N. Srebro, and K. Sridharan, "Better mini-batch algorithms via accelerated gradient methods," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Dec. 2011, pp. 1647–1655.
[34] K. Xiong, Y. Zhang, and S. Wang, "Complex-valued adaptive filtering based on the random Fourier features method," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 67, no. 10, pp. 2284–2288, Oct. 2020.
[35] Y. Xiao et al., "Generalized hyperbolic tangent based random Fourier conjugate gradient filter for nonlinear active noise control," *IEEE/ACM Trans. Audio, Speech, Language Process.*, vol. 31, pp. 619–632, Jan. 2023.
[36] V. Elvira, L. Martino, and P. Closas, "Importance Gaussian quadrature," *IEEE Trans. Signal Process.*, vol. 69, pp. 474–488, Feb. 2021.
[37] D. P. Laurie, "Computation of Gauss-type quadrature formulas," *J. Comput. Appl. Math.*, vol. 127, nos. 1–2, pp. 201–217, Jan. 2001.
[38] S. Jin and B. Andersson, "A note on the accuracy of adaptive Gauss–Hermite quadrature," *Biometrika*, vol. 107, no. 3, pp. 737–744, Feb. 2020.
[39] B. Yang and M. Dai, "Image analysis by Gaussian–Hermite moments," *Signal Process.*, vol. 91, no. 10, pp. 2290–2303, Oct. 2011.
[40] L. Reichel and M. M. Spalević, "A new representation of generalized averaged Gauss quadrature rules," *Appl. Numer. Math.*, vol. 165, pp. 614–619, Jul. 2021.
[41] J. Konečnỳ, J. Liu, P. Richtárik, and M. Takáč, "Mini-batch semi-stochastic gradient descent in the proximal setting," *IEEE J. Sel. Topics Signal Process.*, vol. 10, no. 2, pp. 242–255, Mar. 2016.
[42] C. Liu and M. Jiang, "Robust adaptive filter with lncosh cost," *Signal Process.*, vol. 168, Mar. 2020, Art. no. 107348.
[43] F. Huang, J. Zhang, and S. Zhang, "Mean-square-deviation analysis of probabilistic LMS algorithm," *Digit. Signal Process.*, vol. 92, pp. 26–35, Sep. 2019.
[44] G. Qian and S. Wang, "Generalized complex correntropy: Application to adaptive filtering of complex data," *IEEE Access*, vol. 6, pp. 19113–19120, Apr. 2018.
[45] X. Fan Wang, G.-Q. Zhong, K.-S. Tang, K. F. Man, and Z.-F. Liu, "Generating chaos in Chua's circuit via time-delay feedback," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 48, no. 9, pp. 1151–1156, Sep. 2001.
[46] W. Shi, K. Xiong, and S. Wang, "Multikernel adaptive filters under the minimum Cauchy kernel loss criterion," *IEEE Access*, vol. 7, pp. 120548–120558, Dec. 2019.
[47] H. Zhang, B. Yang, L. Wang, and S. Wang, "General Cauchy conjugate gradient algorithms based on multiple random Fourier features," *IEEE Trans. Signal Process.*, vol. 69, pp. 1859–1873, Mar. 2021.
[48] Q. Zhang, D. Lin, S. Chen, and S. Wang, "Event-triggered quantized kernel least mean square algorithm," in *Proc. IEEE 32nd Int. Workshop Mach. Learn. Signal Process. (MLSP)*, Aug. 2022, pp. 1–6.

Tian Zhou received the B.Eng. degree in electronic and information engineering from Southwest University, Chongqing, China, in 2021, where he is currently pursuing the M.Eng. degree in information and communication engineering with the College of Electronic and Information Engineering. His research interests include adaptive kernel filtering, adaptive signal processing, and active noise control.

Shiyuan Wang (Senior Member, IEEE) received the B.Eng. and M.Eng. degrees in electronic and information engineering from Southwest Normal University, Chongqing, China, in 2002 and 2005, respectively, and the Ph.D. degree in circuit and system from Chongqing University, Chongqing, in 2011. From August 2012 to August 2013, he was a Research Associate with The Hong Kong Polytechnic University, Hong Kong. He is currently a Professor with the College of Electronic and Information Engineering, Southwest University, Chongqing. His research interests include adaptive signal processing, nonlinear dynamics, simultaneous localization and mapping, and bioinformatics.

Junhui Qian (Member, IEEE) received the Ph.D. degree in signal and information processing from the University of Electronic Science and Technology of China, Chengdu, China, in 2018. From 2016 to 2017, he was a Visiting Graduate Researcher with the Electrical Engineering Department, Columbia University, New York, NY, USA. He is currently an Associate Professor with Chongqing University, Chongqing, China. His current research interests include signal processing for MIMO radar and communication systems, artificial olfaction electronic nose, and biomedical and modern signal processing technology.

Yunfei Zheng received the Ph.D. degree in control science and engineering from Xi'an Jiaotong University, Xi'an, China, in 2021. He is currently a Post-Doctoral Researcher with the College of Electronic and Information Engineering, Southwest University, Chongqing, China. He has published more than 20 articles. His current research interests include machine learning and adaptive signal processing.

Qiangqiang Zhang (Graduate Student Member, IEEE) received the B.Eng. degree from the College of Science and Technology, Sanya University, Hainan, China, in 2013, and the Ph.D. degree in electronic and information engineering from Southwest University, Chongqing, China, in 2024. His research interests include adaptive signal processing, optimal estimation, and distributed control.

Yingying Xiao received the B.Eng. and M.Eng. degrees from the College of Electronic and Information Engineering, Southwest University, Chongqing, China, in 2021 and 2024, respectively. Her current research interests include adaptive signal processing, active noise control, and acoustic echo cancellation.